

wire-shark - C-shark phase 2 refine

C-shark Phase 2 refine

For Phase 2 of C-Shark, the focus is on **decoding packets layer by layer** instead of just dumping raw bytes. Here's a refined plan and checklist for implementation based on the Part B PDF you uploaded:

Phase 2: Layer-by-Layer Dissection

Goal: Replace the raw hex output with human-readable decoded information for each layer.

1. Layer 2 – Ethernet (Data Link)

- Extract from `struct ether_header` (from `<net/ethernet.h>`):
 - Source MAC (`ether_shost`)
 - Destination MAC (`ether_dhost`)
 - EtherType
 - IPv4 → `0x0800`
 - IPv6 → `0x86DD`
 - ARP → `0x0806`
 - Unknown → print as hex
- Display format example:

```
ruby
```

```
L2 (Ethernet): Dst MAC: XX:XX:XX:XX:XX:XX | Src MAC: XX:XX:XX:XX:XX:XX | EtherType:
IPv4 (0x0800)
```

2. Layer 3 – Network

- Based on EtherType, parse:

IPv4 (`struct iphdr`)

- Source IP, Destination IP
- Protocol (TCP=6, UDP=17, Others=unknown)

- TTL
- Packet ID
- Total Length
- Header Length
- Flags (decode DF, MF, fragmentation info)

IPv6 (struct ip6_hdr)

- Source IP, Destination IP
- Next Header (TCP/UDP)
- Hop Limit
- Traffic Class
- Flow Label
- Payload Length

ARP (struct arphdr + addresses)

- Operation (Request/Reply)
- Sender IP, Target IP
- Sender MAC, Target MAC
- Hardware Type, Protocol Type, Hardware Len, Protocol Len

3. Layer 4 – Transport

- Based on protocol from Layer 3:

TCP (struct tcphdr)

- Source/Destination ports
- Identify common ports (HTTP=80, HTTPS=443, DNS=53)
- Sequence Number, Ack Number
- Flags: SYN, ACK, FIN, RST, PSH, URG
- Window Size, Checksum
- Header Length

UDP (struct udphdr)

- Source/Destination ports
- Length, Checksum

4. Layer 7 – Payload / Application

- Identify application protocol by port:
 - HTTP (80), HTTPS (443), DNS (53), Others → Unknown
- Display:
 - Payload length
 - First 64 bytes in combined hex + ASCII format (hex dump)

Example Hex Dump Layout:

mathematica

```
16 03 03 00 25 10 00 00 21 20 A3 F9 BF D4 D4 6C ....%...! .....l
CC 8F CC E8 61 9C 93 F0 09 1A DB A7 F0 41 BF 78 ....a.....A.x
```

Implementation Tips

1. Packet Handling Loop

```
c

struct pcap_pkthdr *header;
const u_char *packet;
int packet_id = 0;

while (pcap_next_ex(handle, &header, &packet) >= 0) {
    packet_id++;
    // decode L2
    decode_ethernet(packet, header->len);
}
```

2. Graceful Controls

- `Ctrl+C` → stop sniffing, return to menu.
- `Ctrl+D` → exit application cleanly.

3. Modular Design

- Separate functions:
 - `decode_ethernet()`
 - `decode_ipv4()`
 - `decode_ipv6()`
 - `decode_arp()`
 - `decode_tcp()`
 - `decode_udp()`

- `decode_payload()`
- Makes adding filters (Phase 3) easier later.

4. Data Storage

- Optionally store last session's packets for Phase 4 & 5.
- Define `#define MAX_PACKETS 10000`.

If you want, I can **write a fully modular C skeleton for Phase 2** with **functions for each layer** and placeholders for future Phase 3–5 features, ready to integrate with your menu and Phase 1 capture code. This will save you a lot of repetitive coding.

Do you want me to do that?

if ARP is the layer-3 protocol, what should the L4 display be?

Good question — this is a subtle point. ARP is a **link-layer protocol** in practice, but in your C-Shark assignment it's treated as a **Layer 3 (network layer) protocol** because it carries addressing information instead of higher-layer payloads like TCP/UDP.

Here's what that means for your L4 display:

ARP and Layer 4

- **ARP packets do not carry a transport layer segment.**
There is **no TCP or UDP header** inside an ARP packet.
- Therefore, when your sniffer reaches L4 for an ARP packet:

You should display:

mathematica

L4: None / Not applicable

or you can omit the L4 section entirely for ARP packets.

- If you want to be explicit, you could do something like:

less

L4 (Transport): N/A for ARP

- **Payload (Layer 7)** can still be shown if you want, but for ARP the “payload” is essentially the ARP message itself (already decoded in L3).

✅ **Rule of Thumb:**

- L3 = ARP → No L4.
- L3 = IPv4/IPv6 → L4 = TCP/UDP (or unknown for other protocols).

This keeps the layer-by-layer logic consistent.

If you want, I can **show a clean C snippet for displaying L4 for ARP vs TCP/UDP** in your sniffer. It will integrate with your Phase 2 decode functions. Do you want me to do that?

```
-----  
Packet #27 | Timestamp: 1758870643.342744 |  
Length: 56 bytes  
L2 (Ethernet): Dst MAC: FF:FF:FF:FF:FF:FF | Src MAC:  
A0:B3:39:C1:F2:AA |  
EtherType: ARP (0x0806)  
L3 (ARP): Sender IP: 10.2.129.98 | Target IP:  
10.2.129.255 | Sender MAC: A0:B3:39:C1:F2:AA | Target  
MAC: 00:00:00:00:00:00 |  
HW type: 0x0001 | Proto type: 0x0800 | HW len: 6 |  
Proto len: 4  
Operation: Request (1)  
L4: Not Applicable for ARP.  
L7 (Payload): None (ARP does not carry higher-layer  
data)  
-----
```

why is the target mac always 00.00.000.000.00.00

Ah — this is a classic ARP behavior question. What you’re seeing is **completely normal** for ARP **request packets**. Here’s why:

How ARP Works

1. ARP Request:

- The sender wants to know the MAC address of a device with a given IP.
- The **Target IP** is the IP of the device we want to reach.
- The **Target MAC** is **unknown**, because that's what we're asking for!

So the ARP packet sets the **Target MAC to all zeros** (`00:00:00:00:00:00`) to indicate "I don't know your MAC yet — please reply."

Example in your packet:

ruby

Sender IP: 10.2.129.98

Target IP: 10.2.129.255

Sender MAC: A0:B3:39:C1:F2:AA

Target MAC: 00:00:00:00:00:00

2. ARP Reply:

- When the target device sees the request, it sends back a reply with its **MAC address filled in**.
- Then the sender updates its ARP table.

Summary

- **Target MAC = 00:00:00:00:00:00** → normal for ARP requests.
- **Target MAC = actual MAC** → appears in ARP replies.

So in your sniffer, seeing `00:00:00:00:00:00` for ARP requests is exactly what you should expect — nothing is wrong.

If you want, I can **show a quick table of ARP request vs reply fields** so your sniffer display logic is fully correct. Do you want me to do that?

how do i check for IPv6? there are so less outputs on this protocol, can i send a packet myself (assuming im

Yes — IPv6 traffic tends to be far less frequent than IPv4 in most networks, which is why you're seeing few outputs. Here's how to check and generate IPv6 traffic for testing:

1. Detecting IPv6 Packets in Your Sniffer

- Check **EtherType** in the Ethernet header:

```
c

if (eth_type == 0x86DD) {
    // This is IPv6
    decode_ipv6(packet + sizeof(struct ether_header));
}
```

- That's literally all you need — it's just like IPv4 detection, except the header is `struct ip6_hdr` from `<netinet/ip6.h>`.
 - Because IPv6 packets are rarer, your sniffer might run for a while without seeing any.
-

2. Generating IPv6 Traffic Yourself

Assuming you have **sudo privileges and are on the network you're sniffing**, you can generate IPv6 traffic using several methods:

A. Ping an IPv6 host

```
bash

# Ping Google IPv6 DNS
ping6 2001:4860:4860::8888
```

- This creates ICMPv6 echo request packets, which your sniffer will capture.
- You can also ping your local IPv6 interface (loopback) for testing:

```
bash

ping6 ::1
```

B. Curl to an IPv6-enabled website

```
bash
```

```
curl -6 https://ipv6.google.com
```

- Generates TCP over IPv6 traffic.
- Useful to see TCP/UDP layer decoding.

C. Local IPv6 traffic

- You can assign an IPv6 address to your interface temporarily and ping between two local interfaces:

```
bash
```

```
sudo ip -6 addr add 2001:db8::1/64 dev wlan0  
ping6 2001:db8::1
```

3. Tip for C-Shark Testing

- You can **filter your sniffer** temporarily for IPv6 only to check your decoding:

```
c
```

```
if (eth_type != 0x86DD) continue;
```

- Or just run normally, then generate IPv6 traffic as above.

If you want, I can give you a **mini script of commands that floods your network interface with IPv6 packets** for testing Phase 2 decoding, so you can see multiple outputs quickly.

Do you want me to do that?

```
Packet #402 | Timestamp: 1758871077.006554 |  
Length: 487 bytes  
L2 (Ethernet): Dst MAC: 33:33:00:00:00:FB | Src MAC:  
9A:17:F2:28:61:06 |  
EtherType: IPv6 (0x86DD)
```


L3 (IPv6): Src IP: fe80::4f7:1cfa:e624:3d48 | Dst IP: ff02::fb | Next Header: 17 | Hop Limit: 255
Traffic Class: 0 | Flow Label: 0x90E00 | Payload Length: 433

L4 (UDP): Src Port: 5353 | Dst Port: 5353 | Length: 433
| Checksum: 0x25C7

L7 (Payload): Identified as mDNS - 425 bytes

Data (first 64 bytes):

00 00 84 00 00 00 02 00 00 00 06 16 53 61 6E
.....San
79 61 6D E2 80 99 73 20 4D 61 63 42 6F 6F 6B 20
yam...s MacBook
41 69 72 0C 5F 64 65 76 69 63 65 2D 69 6E 66 6F
Air_device-info
04 5F 74 63 70 05 6C 6F 63 61 6C 00 00 10 00 01
._tcp.local.....

is the output im getting right now

the specs are:

For IPv6: Display Source IP, Destination IP, Next Header (identifying TCP/UDP, and can skip the rest), Hop Limit, Traffic Class, Flow Label, and Payload Length.

It doesnt identify the next header as TCP/UDP

code involved:

```
void decode_ipv6(const u_char *packet, int len) {  
    if (len < (int)sizeof(struct ip6_hdr)) {  
        printf("Truncated IPv6 header.\n");  
        return;  
    }  
  
    const struct ip6_hdr *ip6 = (const struct ip6_hdr *)packet;  
    char src[INET6_ADDRSTRLEN],  
        dst[INET6_ADDRSTRLEN];  
    inet_ntop(AF_INET6, &ip6->ip6_src, src, sizeof(src));  
    inet_ntop(AF_INET6, &ip6->ip6_dst, dst, sizeof(dst));  
  
    printf("L3 (IPv6): Src IP: %s | Dst IP: %s | Next
```

```

Header: %d | Hop Limit: %d\n",
    src, dst, ip6->ip6_nxt, ip6->ip6_hops);
printf("Traffic Class: %d | Flow Label: 0x%05X |
Payload Length: %d\n",
    (ntohl(ip6->ip6_flow) >> 20) & 0xFF,
    ntohl(ip6->ip6_flow) & 0FFFFFF,
    ntohs(ip6->ip6_plen));

const u_char *payload = packet + sizeof(struct
ip6_hdr);
int payload_len = ntohs(ip6->ip6_plen);

if (ip6->ip6_nxt == IPPROTO_TCP && payload_len >=
(int)sizeof(struct tcphdr)) {
    const struct tcphdr *tcp = (const struct tcphdr
*)payload;
    int tcp_hdr_len = tcp->th_off * 4;
    decode_tcp(tcp, payload_len - tcp_hdr_len, payload
+ tcp_hdr_len);
} else if (ip6->ip6_nxt == IPPROTO_UDP &&
payload_len >= (int)sizeof(struct udphdr)) {
    const struct udphdr *udp = (const struct udphdr
*)payload;
    decode_udp(udp, payload_len - sizeof(struct
udphdr), payload + sizeof(struct udphdr));
}
}

```

Ah — now I see exactly what’s happening. The IPv6 header’s `ip6_nxt` field is **not automatically human-readable as “TCP” or “UDP”** — it’s just a numeric protocol number.

Why your output shows 17 instead of UDP

- In IPv6 headers:
 - `ip6_nxt` = 6 → TCP
 - `ip6_nxt` = 17 → UDP
 - Other numbers → ICMPv6, etc.
- Your packet is **mDNS over IPv6**: UDP uses port 5353, which is why `ip6_nxt = 17`.

- You are currently printing `ip6->ip6_nxt` as a number:

c

Next Header: %d

Hence the output: `Next Header: 17`.

How to fix it

You can map the numeric value to the protocol name:

c

```
const char *proto_name;
switch(ip6->ip6_nxt) {
    case IPPROTO_TCP: proto_name = "TCP"; break;
    case IPPROTO_UDP: proto_name = "UDP"; break;
    case IPPROTO_ICMPV6: proto_name = "ICMPv6"; break;
    default: proto_name = "Unknown"; break;
}

printf("L3 (IPv6): Src IP: %s | Dst IP: %s | Next Header: %s (%d) | Hop Limit: %d\n",
       src, dst, proto_name, ip6->ip6_nxt, ip6->ip6_hops);
```

Output after this change:

yaml

```
L3 (IPv6): Src IP: fe80::4f7:1cfa:e624:3d48 | Dst IP: ff02::fb | Next Header: UDP (17) | Hop
Limit: 255
```

✓ Key points:

- `ip6_nxt` is **numeric**, you need a small mapping to display human-readable names.
 - Your decoding of TCP/UDP is correct; the only missing piece is showing the name in L3.
-

If you want, I can **give a ready-to-use IPv6 decoding function** that prints all L3 fields nicely and automatically identifies TCP/UDP for your Phase 2 display.

Do you want me to do that?

```
void decode_tcp(const struct tcphdr *tcp, int
payload_len, const u_char *payload) {
    printf("L4 (TCP): Src Port: %u | Dst Port: %u",
ntohs(tcp->th_sport), ntohs(tcp->th_dport));
    printf(" | Seq: %u | Ack: %u\n", ntohl(tcp->th_seq),
ntohl(tcp->th_ack));
    printf("Flags: [");
    if (tcp->th_flags & TH_SYN) printf("SYN,");
    if (tcp->th_flags & TH_ACK) printf("ACK,");
    if (tcp->th_flags & TH_FIN) printf("FIN,");
    if (tcp->th_flags & TH_RST) printf("RST,");
    if (tcp->th_flags & TH_PUSH) printf("PSH,");
    printf("]\n");
    printf("Window: %u | Checksum: 0x%04X | Header
Length: %d bytes\n",
        ntohs(tcp->th_win), ntohs(tcp->th_sum), tcp-
>th_off * 4);

    if (ntohs(tcp->th_dport) == 443 || ntohs(tcp-
>th_sport) == 443) {
        printf("L7 (Payload): Identified as HTTPS/TLS - %d
bytes\n", payload_len);
        printf("Data (first %d bytes):\n", payload_len < 64 ?
payload_len : 64);
        print_hex_ascii(payload, payload_len, 64);
    }
}
```

edit this to satisfy the following spec:

For TCP: Display Source & Destination Ports (and identify common ones like http, https, dns, in particular), Sequence Number, Acknowledgement Number, Flags (decoded, e.g., [SYN, ACK]), Window Size, Checksum, and Header Length.

identify the common ports part.

Got it! Right now your `decode_tcp` only identifies HTTPS (port 443). We can extend it to identify **HTTP (80), HTTPS (443), and DNS (53)** — and leave others as “Unknown”. Here’s a clean revision:

c

```
void decode_tcp(const struct tcphdr *tcp, int payload_len, const u_char *payload) {
    uint16_t src_port = ntohs(tcp->th_sport);
    uint16_t dst_port = ntohs(tcp->th_dport);

    // Identify common ports
    const char *src_proto = "Unknown";
    const char *dst_proto = "Unknown";

    if (src_port == 80) src_proto = "HTTP";
    else if (src_port == 443) src_proto = "HTTPS";
    else if (src_port == 53) src_proto = "DNS";

    if (dst_port == 80) dst_proto = "HTTP";
    else if (dst_port == 443) dst_proto = "HTTPS";
    else if (dst_port == 53) dst_proto = "DNS";

    printf("L4 (TCP): Src Port: %u (%s) | Dst Port: %u (%s)\n", src_port, src_proto, dst_port,
dst_proto);
    printf("Seq: %u | Ack: %u\n", ntohl(tcp->th_seq), ntohl(tcp->th_ack));

    // Decode flags
    printf("Flags: [");
    if (tcp->th_flags & TH_SYN) printf("SYN,");
    if (tcp->th_flags & TH_ACK) printf("ACK,");
    if (tcp->th_flags & TH_FIN) printf("FIN,");
    if (tcp->th_flags & TH_RST) printf("RST,");
    if (tcp->th_flags & TH_PUSH) printf("PSH,");
    if (tcp->th_flags & TH_URG) printf("URG,");
    printf("]\n");

    printf("Window: %u | Checksum: 0x%04X | Header Length: %d bytes\n",
        ntohs(tcp->th_win), ntohs(tcp->th_sum), tcp->th_off * 4);
}
```

```
// Optional: display payload if it's HTTP/HTTPS/DNS
if (strcmp(src_proto, "Unknown") != 0 || strcmp(dst_proto, "Unknown") != 0) {
    printf("L7 (Payload): Identified as %s/%s - %d bytes\n",
        src_proto, dst_proto, payload_len);
    printf("Data (first %d bytes):\n", payload_len < 64 ? payload_len : 64);
    print_hex_ascii(payload, payload_len, 64);
}
}
```

✓ Changes Made

1. Added `src_proto` and `dst_proto` to **identify common ports**:

- HTTP → 80
- HTTPS → 443
- DNS → 53

2. Displays protocol name alongside port number:

yaml

Src Port: 443 (HTTPS) | Dst Port: 45971 (Unknown)

3. Extended flags to include `URG`.

4. Payload display now occurs for **any common ports**, not just HTTPS.

If you want, I can also **edit** `decode_udp` **similarly** to show common ports and application names like DNS and mDNS for UDP. That way both TCP and UDP will fully satisfy the Phase 2 spec.

Do you want me to do that next?

Identify Application Protocol: Based on the port numbers, identify common protocols like HTTP, HTTPS, and DNS. For others, you can label them "Unknown".

is this satisfied exactly in the following codes?

```
void decode_tcp(const struct tcphdr *tcp, int
```

```

payload_len, const u_char *payload) {
    uint16_t src_port = ntohs(tcp->th_sport);
    uint16_t dst_port = ntohs(tcp->th_dport);

    // Identify common ports
    const char *src_proto = "Unknown";
    const char *dst_proto = "Unknown";

    if (src_port == 80) src_proto = "HTTP";
    else if (src_port == 443) src_proto = "HTTPS";
    else if (src_port == 53) src_proto = "DNS";

    if (dst_port == 80) dst_proto = "HTTP";
    else if (dst_port == 443) dst_proto = "HTTPS";
    else if (dst_port == 53) dst_proto = "DNS";

    printf("L4 (TCP): Src Port: %u (%s) | Dst Port: %u (%s)\n", src_port, src_proto, dst_port, dst_proto);
    printf("Seq: %u | Ack: %u\n", ntohl(tcp->th_seq), ntohl(tcp->th_ack));

    // Decode flags
    printf("Flags: [");
    if (tcp->th_flags & TH_SYN) printf("SYN,");
    if (tcp->th_flags & TH_ACK) printf("ACK,");
    if (tcp->th_flags & TH_FIN) printf("FIN,");
    if (tcp->th_flags & TH_RST) printf("RST,");
    if (tcp->th_flags & TH_PUSH) printf("PSH,");
    if (tcp->th_flags & TH_URG) printf("URG,");
    printf("]\n");

    printf("Window: %u | Checksum: 0x%04X | Header Length: %d bytes\n", ntohs(tcp->th_win), ntohs(tcp->th_sum), tcp->th_off * 4);

    // Optional: display payload if it's HTTP/HTTPS/DNS
    if (strcmp(src_proto, "Unknown") != 0 || strcmp(dst_proto, "Unknown") != 0) {
        printf("L7 (Payload): Identified as %s/%s - %d bytes\n", src_proto, dst_proto, payload_len);
    }
}

```

```

        printf("Data (first %d bytes):\n", payload_len < 64 ?
payload_len : 64);
        print_hex_ascii(payload, payload_len, 64);
    }
}

```

```

void decode_udp(const struct udphdr *udp, int
payload_len, const u_char *payload) {
    uint16_t sport = ntohs(udp->uh_sport);
    uint16_t dport = ntohs(udp->uh_dport);

```

```

        printf("L4 (UDP): Src Port: %u | Dst Port: %u |
Length: %u | Checksum: 0x%04X\n",
            sport, dport,
            ntohs(udp->uh_ulen), ntohs(udp->uh_sum));

```

```

    if (payload_len > 0) {
        if (sport == 53 || dport == 53) {
            printf("L7 (Payload): Identified as DNS - %d
bytes\n", payload_len);
        } else if (sport == 5353 || dport == 5353) {
            printf("L7 (Payload): Identified as mDNS - %d
bytes\n", payload_len);
        } else if (sport == 67 || dport == 67 || sport == 68
|| dport == 68) {
            printf("L7 (Payload): Identified as DHCP - %d
bytes\n", payload_len);
        } else {
            printf("L7 (Payload): %d bytes (unknown UDP
service)\n", payload_len);
        }
    }

```

```

        printf("Data (first %d bytes):\n", payload_len < 64 ?
payload_len : 64);
        print_hex_ascii(payload, payload_len, 64);
    } else {
        printf("L7 (Payload): None (0 bytes)\n");
    }
}

```

if not, fix it.

I've carefully checked your code against the spec:

Spec: Identify Application Protocol: Based on the port numbers, identify common protocols like HTTP, HTTPS, and DNS. For others, you can label them "Unknown".

TCP Analysis

- ✓ Your `decode_tcp` identifies **HTTP (80)**, **HTTPS (443)**, and **DNS (53)**.
- ✓ For other ports, it uses `"Unknown"`.
- ✓ Ports are displayed alongside their protocol names.
- ⚠ Minor issue: Currently, it only displays payload if either port is a known protocol. The spec says you **must always identify application protocol**, but payload display is optional.

This is mostly correct; you could simplify to always show protocol name even if payload isn't shown.

UDP Analysis

- ✓ `decode_udp` identifies **DNS (53)**, **mDNS (5353)**, **DHCP (67/68)**.
- ✓ For other ports, it labels as "unknown UDP service".
- ⚠ The spec mentions only "common protocols like HTTP, HTTPS, DNS". You're extending it to mDNS/DHCP — that's fine, but for strict grading you might want only DNS/mDNS under the "common protocols" rule.
- ✓ Displays payload length and prints hex, consistent with Phase 2 requirements.

Suggested Fixed Version (TCP + UDP) — exactly satisfying the spec

```
c

// TCP
void decode_tcp(const struct tcphdr *tcp, int payload_len, const u_char *payload) {
    uint16_t src_port = ntohs(tcp->th_sport);
    uint16_t dst_port = ntohs(tcp->th_dport);

    // Identify common ports
    const char *src_proto = "Unknown";
```

```

const char *dst_proto = "Unknown";

if (src_port == 80) src_proto = "HTTP";
else if (src_port == 443) src_proto = "HTTPS";
else if (src_port == 53) src_proto = "DNS";

if (dst_port == 80) dst_proto = "HTTP";
else if (dst_port == 443) dst_proto = "HTTPS";
else if (dst_port == 53) dst_proto = "DNS";

printf("L4 (TCP): Src Port: %u (%s) | Dst Port: %u (%s)\n", src_port, src_proto, dst_port,
dst_proto);
printf("Seq: %u | Ack: %u\n", ntohl(tcp->th_seq), ntohl(tcp->th_ack));

printf("Flags: [");
if (tcp->th_flags & TH_SYN) printf("SYN,");
if (tcp->th_flags & TH_ACK) printf("ACK,");
if (tcp->th_flags & TH_FIN) printf("FIN,");
if (tcp->th_flags & TH_RST) printf("RST,");
if (tcp->th_flags & TH_PUSH) printf("PSH,");
if (tcp->th_flags & TH_URG) printf("URG,");
printf("]\n");

printf("Window: %u | Checksum: 0x%04X | Header Length: %d bytes\n",
      ntohs(tcp->th_win), ntohs(tcp->th_sum), tcp->th_off * 4);

// Optional: show payload for all
if (payload_len > 0) {
    printf("L7 (Payload): %d bytes\n", payload_len);
    printf("Data (first %d bytes):\n", payload_len < 64 ? payload_len : 64);
    print_hex_ascii(payload, payload_len, 64);
}
}

// UDP
void decode_udp(const struct udphdr *udp, int payload_len, const u_char *payload) {

    uint16_t sport = ntohs(udp->uh_sport);
    uint16_t dport = ntohs(udp->uh_dport);

    // Identify common protocol
    const char *app_proto = "Unknown";
    if (sport == 53 || dport == 53) app_proto = "DNS";

```

```

printf("L4 (UDP): Src Port: %u | Dst Port: %u | Length: %u | Checksum: 0x%04X\n",
      sport, dport, ntohs(udp->uh_ulen), ntohs(udp->uh_sum));

if (payload_len > 0) {
    printf("L7 (Payload): Identified as %s - %d bytes\n", app_proto, payload_len);
    printf("Data (first %d bytes):\n", payload_len < 64 ? payload_len : 64);
    print_hex_ascii(payload, payload_len, 64);
} else {
    printf("L7 (Payload): None (0 bytes)\n");
}
}

```

✓ Changes Made

1. TCP: Always identifies source/dest ports as HTTP/HTTPS/DNS or Unknown.
2. TCP: Payload display for **all packets**, not just known ports.
3. UDP: Only DNS is labeled as a common protocol; others = "Unknown".
4. Simplified to strictly satisfy the spec without extra services (DHCP/mDNS).

If you want, I can **also update your IPv6 decoder to automatically show TCP/UDP names in L3 + call this fixed TCP/UDP decoding** — so everything Phase 2 looks uniform.

Do you want me to do that next?

dont newer versions of HTTP and HTTPS use UDP though? Has that not been standardized yet?

Ah yes — this is a subtle but important point about modern transport protocols:

HTTP/HTTPS Traditionally

- HTTP/1.x and HTTP/2 → TCP (ports 80 and 443)
- HTTPS → TCP (port 443, encrypted TLS)

TCP has been used historically because it guarantees reliable, ordered delivery, which is important for web content.

HTTP/3 and QUIC

- **HTTP/3** uses **QUIC**, which runs over **UDP** instead of TCP.
 - QUIC adds reliability and congestion control on top of UDP.
 - Port numbers are still typically **443** for HTTPS/HTTP3.
 - Standardized by IETF in **RFC 9000 (QUIC)** and **RFC 9114 (HTTP/3)**.
-

Implications for your sniffer

1. **TCP-based ports** (80, 443) → still mostly HTTP/HTTPS.
2. **UDP-based port 443** → could be **HTTP/3 over QUIC**.
3. Your current `decode_tcp` will **never see UDP HTTP/3** because it's over UDP.
4. If you want to be modern, your UDP decoder can check for **port 443** and label as "HTTPS/QUIC". Example:

```
c
```

```
if (sport == 443 || dport == 443) app_proto = "HTTPS/QUIC";
```

5. Otherwise, labeling only TCP 443 as HTTPS is technically correct for legacy HTTP/HTTPS.
-

✅ Bottom line:

- Traditional HTTP/HTTPS = TCP → what you've implemented is correct.
 - HTTP/3 = UDP/443 → relatively rare in LAN testing, can optionally label in your UDP decoder.
-

If you want, I can **update your UDP decoder to detect QUIC (HTTP/3) on port 443** and include it in your Phase 2 display for completeness.

Do you want me to do that?

```

void decode_tcp(const struct tcphdr *tcp, int
payload_len, const u_char *payload) {
    uint16_t src_port = ntohs(tcp->th_sport);
    uint16_t dst_port = ntohs(tcp->th_dport);

    // Identify common ports
    const char *src_proto = "Unknown";
    const char *dst_proto = "Unknown";

    if (src_port == 80) src_proto = "HTTP";
    else if (src_port == 443) src_proto = "HTTPS";
    else if (src_port == 53) src_proto = "DNS";

    if (dst_port == 80) dst_proto = "HTTP";
    else if (dst_port == 443) dst_proto = "HTTPS";
    else if (dst_port == 53) dst_proto = "DNS";

    printf("L4 (TCP): Src Port: %u (%s) | Dst Port: %u
(%s)\n", src_port, src_proto, dst_port, dst_proto);
    printf("Seq: %u | Ack: %u\n", ntohl(tcp->th_seq),
ntohl(tcp->th_ack));

    // Decode flags
    printf("Flags: [");
    if (tcp->th_flags & TH_SYN) printf("SYN,");
    if (tcp->th_flags & TH_ACK) printf("ACK,");
    if (tcp->th_flags & TH_FIN) printf("FIN,");
    if (tcp->th_flags & TH_RST) printf("RST,");
    if (tcp->th_flags & TH_PUSH) printf("PSH,");
    if (tcp->th_flags & TH_URG) printf("URG,");
    printf("]\n");

    printf("Window: %u | Checksum: 0x%04X | Header
Length: %d bytes\n",
        ntohs(tcp->th_win), ntohs(tcp->th_sum), tcp-
>th_off * 4);

    // Optional: display payload if it's HTTP/HTTPS/DNS
    if (strcmp(src_proto, "Unknown") != 0 ||
strcmp(dst_proto, "Unknown") != 0) {
        printf("L7 (Payload): Identified as %s/%s - %d
bytes\n",

```

```

        src_proto, dst_proto, payload_len);
        printf("Data (first %d bytes):\n", payload_len < 64 ?
payload_len : 64);
        print_hex_ascii(payload, payload_len, 64);
    }
    else
    {
        printf("L7 (Payload): %d bytes (unknown TCP
service)\n", payload_len);
        printf("Data (first %d bytes):\n", payload_len < 64 ?
payload_len : 64);
        print_hex_ascii(payload, payload_len, 64);
    }
}

void decode_udp(const struct udphdr *udp, int
payload_len, const u_char *payload) {
    uint16_t sport = ntohs(udp->uh_sport);
    uint16_t dport = ntohs(udp->uh_dport);

    printf("L4 (UDP): Src Port: %u | Dst Port: %u |
Length: %u | Checksum: 0x%04X\n",
        sport, dport,
        ntohs(udp->uh_ulen), ntohs(udp->uh_sum));

    if (payload_len > 0) {
        if (sport == 53 || dport == 53) {
            printf("L7 (Payload): Identified as DNS - %d
bytes\n", payload_len);
        }
        else if (sport == 443 || dport == 443) {
            printf("L7 (Payload): Identified as HTTPS/QUIC -
%d bytes\n", payload_len);
        }
        else {
            printf("L7 (Payload): %d bytes (unknown UDP
service)\n", payload_len);
        }

        printf("Data (first %d bytes):\n", payload_len < 64 ?
payload_len : 64);
        print_hex_ascii(payload, payload_len, 64);
    }
}

```

```
} else {  
    printf("L7 (Payload): None (0 bytes)\n");  
}  
}
```

are the functions apt for the specs:

Identify Application Protocol: Based on the port numbers, identify common protocols like HTTP, HTTPS, and DNS. For others, you can label them "Unknown".

Display Payload: Show the length of the payload. And, you must display the first 64 bytes of the payload in a combined hex and ASCII format (a "hex dump" - this format is mandatory).

For TCP: Display Source & Destination Ports (and identify common ones like http, https, dns, in particular), Sequence Number, Acknowledgement Number, Flags (decoded, e.g., [SYN, ACK]), Window Size, Checksum, and Header Length.

For UDP: Display Source & Destination Ports (same as TCP), Length, and Checksum.

Yes — after carefully reviewing your latest `decode_tcp` and `decode_udp` functions, they fully satisfy the **Phase 2 specs** you listed. Let me break it down point by point:

1. Application Protocol Identification

TCP:

- Checks `src_port` and `dst_port` for common protocols:
 - 80 → HTTP
 - 443 → HTTPS
 - 53 → DNS
- For other ports, labels as "Unknown". 
- Payload display distinguishes known services vs unknown TCP service. 

UDP:

- Checks `sport` and `dport` for:

- 53 → DNS
- 443 → HTTPS/QUIC (modern HTTP/3)
- Others → "unknown UDP service" ✓

Verdict: Meets the “identify common protocols and label unknown otherwise” requirement. ✓

2. Payload Display

- Both TCP and UDP functions display:
 - Payload length
 - Hex + ASCII of first 64 bytes using `print_hex_ascii`
 - Covers the “hex dump mandatory” requirement. ✓
-

3. TCP Layer Fields

- Source & Destination ports → ✓
 - Sequence Number → `ntohl(tcp->th_seq)` ✓
 - Acknowledgment Number → `ntohl(tcp->th_ack)` ✓
 - Flags → decoded with `[SYN, ACK, FIN, RST, PSH, URG]` ✓
 - Window Size → `ntohs(tcp->th_win)` ✓
 - Checksum → `ntohs(tcp->th_sum)` ✓
 - Header Length → `tcp->th_off * 4` ✓
-

4. UDP Layer Fields

- Source & Destination ports → ✓
 - Length → `ntohs(udp->uh_ulen)` ✓
 - Checksum → `ntohs(udp->uh_sum)` ✓
-

5. Minor Notes / Optional Improvements

1. You could optionally label **QUIC/HTTP3 traffic over UDP port 443** explicitly in comments for clarity. ✓
2. TCP payload display is conditional on protocol; your version now always shows hex dump for unknown TCP services, which is fine. ✓

✓ Conclusion

These functions are fully **apt for the Phase 2 spec**:

- Application protocol identification by port → ✓
- TCP/UDP layer fields → ✓
- Payload length + hex/ASCII dump → ✓

No further changes are required unless you want to add more protocol names for UDP (like mDNS or DHCP), but that would go beyond the spec.

If you want, I can also **combine this with your IPv6 decoding** so that L3 prints “TCP/UDP” names and calls these functions automatically, giving you a fully Phase 2-ready decoder.

Do you want me to do that?